



Gestión de Memoria

Cátedra: Organización del Computador II
Lic. Juan Pablo Moreno



Contenido

1. Espacio Físico
2. Espacio Lógico
3. Modos de Operación
4. Modelos de Memoria
5. GDT y LDT



Espacio físico

- IA-32 organiza la memoria como una secuencia de bytes.
- Se direcciona a través de su Bus de Direcciones.
- La memoria conectada a este, se la denomina memoria física.
- Se denomina direcciones físicas al espacio de direcciones que pueden
- volcarse sobre este bus.



Espacio lógico

Intel definió organizar el espacio de direcciones en segmentos. El compromiso de compatibilidad ató a las siguientes generaciones de procesadores.

- 4 registros de segmento para almacenar hasta 4 selectores de segmento.
- Registros de 16 bits, los segmentos tienen a lo sumo 64K de tamaño.
- Expresión de las direcciones en el modelo de programación mediante dos valores:
 - Identificador del segmento en el que se encuentra la variable o la instrucción que se desea direccionar.
 - Desplazamiento, offset, o dirección efectiva a partir del inicio de ese segmento en donde se encuentra efectivamente.



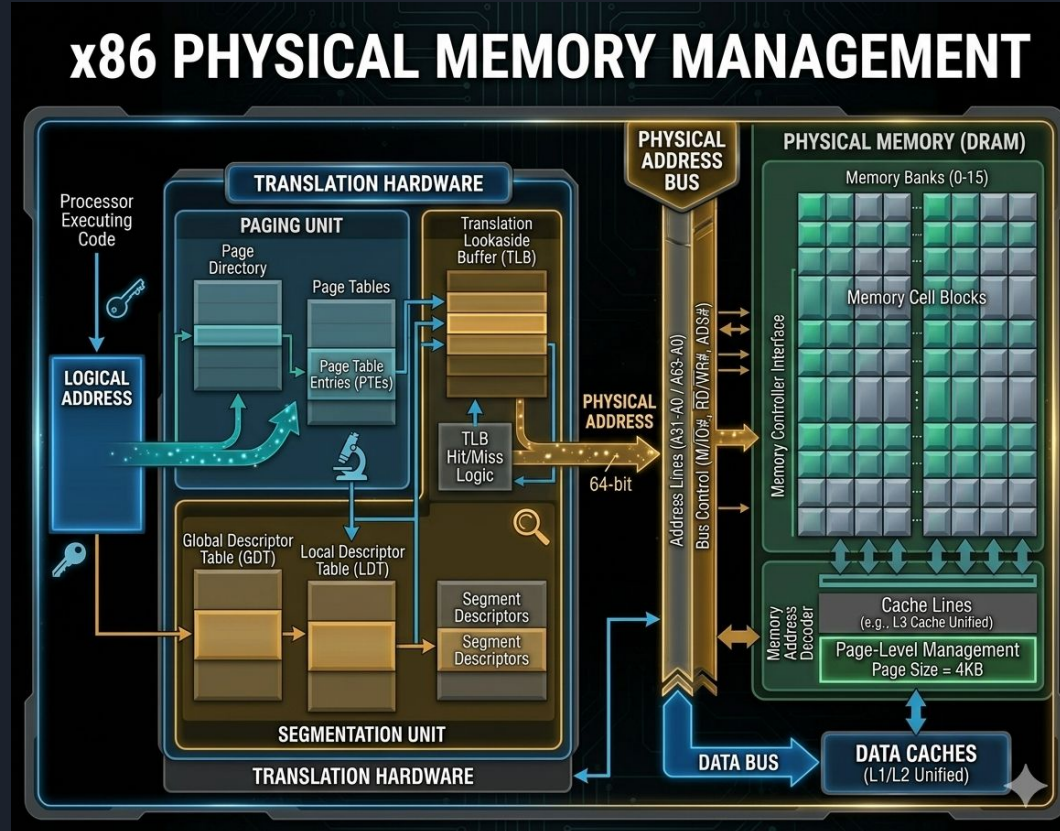
Espacio lógico

A una dirección de memoria expresada en términos de los recursos de la arquitectura del procesador la llamaremos dirección lógica.

Id.Segmento:Offset

- Estos dos valores por si solos no tienen significado físico.
- Para lograr a partir de ellos identificar la dirección física de la variable o instrucción el procesador debe realizar una serie de operaciones.

Gestión de Memoria x86





Modo de operación

- **Modo Protegido:** es el estado nativo del procesador. Tiene la capacidad de ejecutar directamente el software 8086 "modo de dirección real" en un entorno protegido y multitarea.
- **Modo Real:** implementa el entorno de trabajo del 8086 con extensiones. El procesador se coloca en modo de dirección real después del encendido o reinicio.
- **System management mode (SMM):** proporciona mecanismos para implementar funciones específicas sobre la plataforma, como la administración de energía o seguridad.
- **Modo de compatibilidad:** permite la mayoría de las versiones anteriores de 16 bits y las aplicaciones de 32 bits se ejecutan sin volver a compilar en un sistema operativo de 64 bits.
- **Modo de 64 bits:** permite que un sistema operativo de 64 bits ejecute aplicaciones funciones escritas para acceder al espacio de direcciones lineales de 64 bits. Extiende un número de registros de propósito general y registros de extensión SIMD.



Modelos de memoria

Flat Model: la memoria aparece para un programa como un espacio de direcciones único y continuo. El espacio se llama espacio de direcciones lineal. El código, los datos y las pilas están contenidos en este espacio de direcciones.

Real Model: utiliza una implementación específica de memoria segmentada en la que el espacio de direcciones lineal para el programa y el sistema operativo consta de una serie de segmentos de hasta 64 KBytes de tamaño cada uno.

Segmented memory: la memoria aparece para un programa como un grupo de espacios de direcciones independientes llamados segmentos. El código, los datos y el stack suelen estar contenidos en segmentos separados. Para leer un byte en un segmento, un programa emite una dirección lógica. Consiste en un selector de segmento y un desplazamiento. El selector de segmento identifica el segmento al que se debe acceder y el desplazamiento identifica un byte en el espacio de direcciones del segmento.

Modelos de memoria

MODERN MEMORY ARCHITECTURE MODELS

Flat Model

Linear Address

Linear
Address
Space

Segmented Model

Logical
Address

Offset (effective address)

Segment Selector

Segments

Linear
Address
Space

Real-Address Mode Model

Logical
Address

Offset

Segment Selector

Linear Address
Space Divided
into Equal
Sized Segments

* The linear address space
can be paged when using the
flat or segmented model.



Jerarquía de Modos de Operación y Modelos de Memoria (x86-64)

Modo de Operación	Modelo de Memoria	Capacidad de Direccionamiento	Propósito Principal
Real	Segmentado Fijo	1 MB	Arranque y compatibilidad legacy.
Protegido	Plano / Segmentado	4 GB	Ejecución estándar de 32 bits.
Long Mode	Plano	2^{64} (Teórico)	Ejecución nativa moderna de 64 bits.
Compatibilidad	Plano / Segmentado	4 GB	Ejecutar Apps 32-bit en SO 64-bit.
SMM	Segmentado (SMRAM)	Total (Físico)	Gestión de hardware y BIOS/UEFI.

Registros de propósito general

GENERAL-PURPOSE REGISTERS

31	EAX	0
	EBX	
	ECX	
	EDX	
	ESI	
	EDI	
	EBP	
	ESP	

SEGMENT REGISTERS

15	CS	0
	DS	
	SS	
	ES	
	FS	
	GS	

PROGRAM STATUS & CONTROL REGISTER

31	CF	ZF	ZF	EFLAGS	CF	CF	0
----	----	----	----	--------	----	----	---

INSTRUCTION POINTER

31	EIP	0
----	-----	---

REGISTER FUNCTIONS & PURPOSES

EAX – Accumulator for operands and results data 

EBX – Pointer to data in the DS segment 

ECX – Counter for string and loop operations 

EDX – I/O pointer 

ESI – Pointer to data in the segment pointed to by the DS register; source pointer for string operations 

EDI – Pointer to data (or destination) in the segment pointed to by the ES register; destination pointer for string operations

ESP – Stack pointer (in the SS segment) 

EBP – Pointer to data on the stack (in the SS segment) 

Ejecución de una Función (CALL/RET)

GENERAL-PURPOSE REGISTERS



SEGMENT REGISTERS



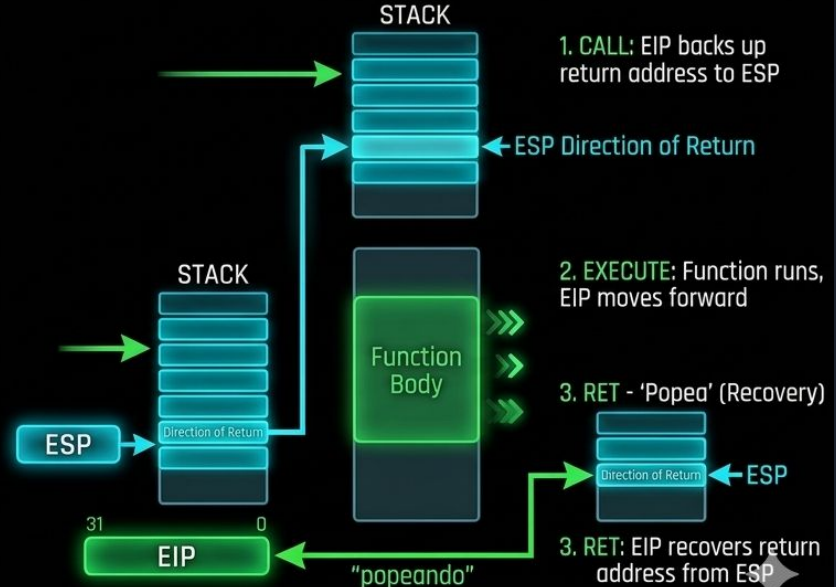
PROGRAM STATUS & CONTROL REGISTER



INSTRUCTION POINTER



EIP vs. ESP INTERACTION: CALL / RET FLOW

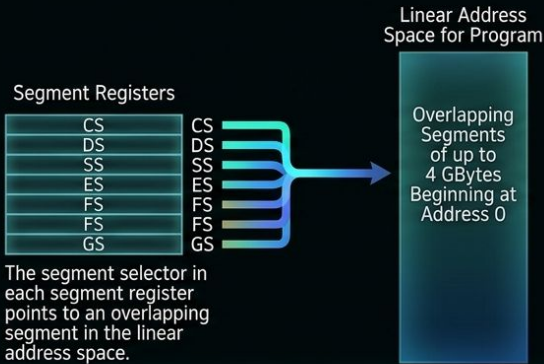


Registros de segmentos

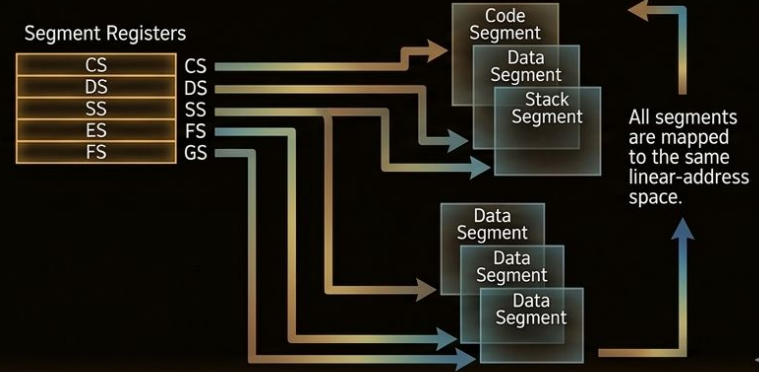
Un selector de segmento es un puntero especial que identifica un segmento en la memoria.

COMPARATIVE SEGMENTATION MODELS: FLAT VS. SEGMENTED

FLAT MEMORY MODEL

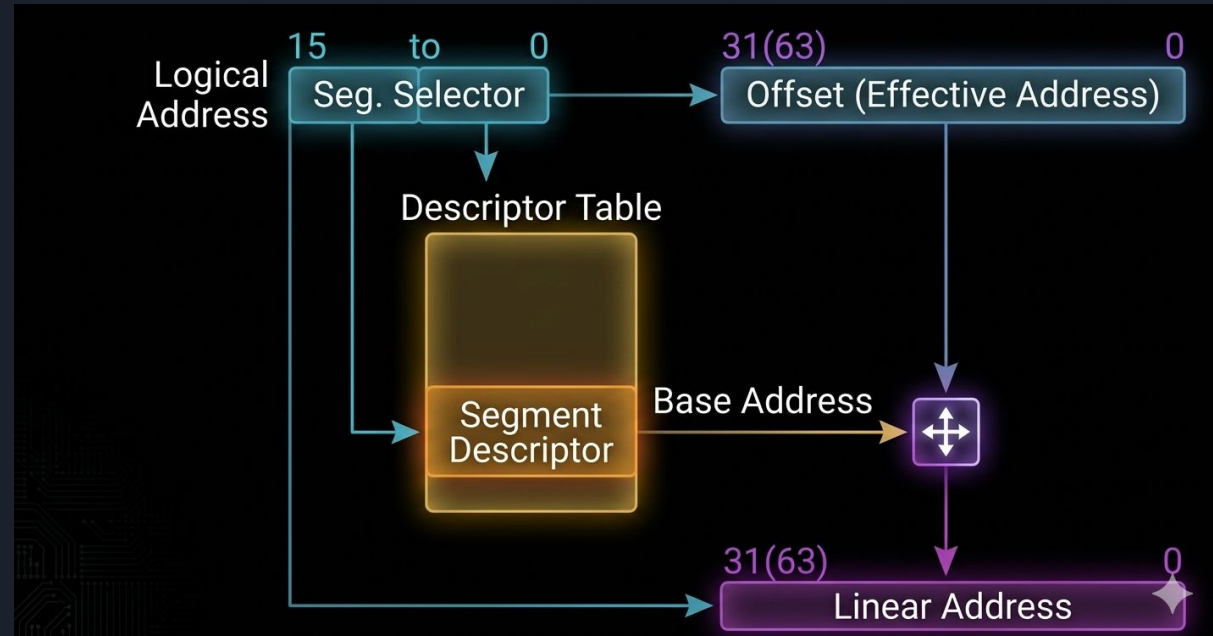


SEGMENTED MEMORY MODEL



Traducción de direcciones Lógicas

En modo protegido, el procesador utiliza dos etapas de traducción de direcciones para llegar a una dirección física: traducción de direcciones lógicas y paginación de espacio de direcciones lineales.





Traducción de direcciones Lógicas

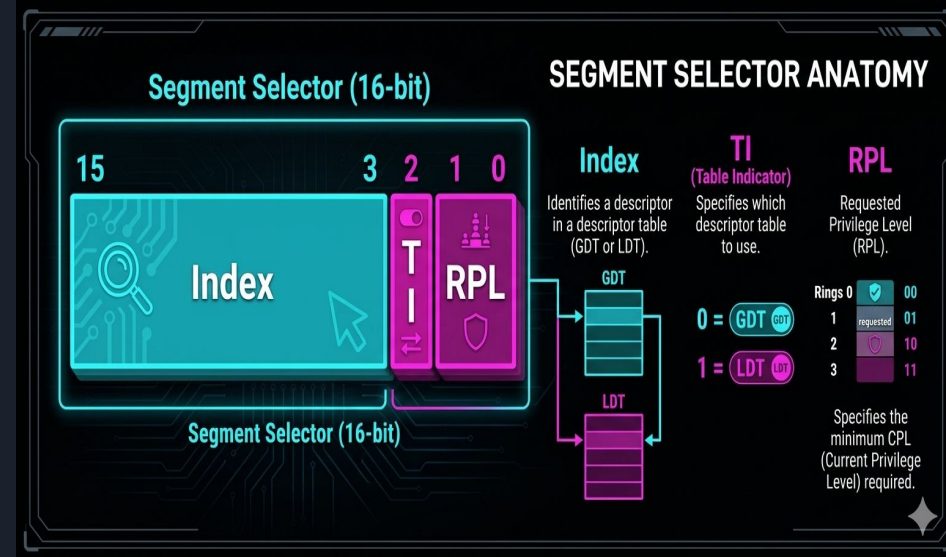
1. Utiliza el desplazamiento en el selector de segmento para localizar el descriptor de segmento en el Global Descriptor Table (GDT modo 64-bits) o Local Descriptor Table (LDT modo protegido) y lo lee en el procesador.
2. Examina el descriptor de segmento para verificar los derechos de acceso y el rango del mismo para asegurar que el segmento es accesible y el desplazamiento está dentro de los límites del segmento.
3. Agrega la dirección base del segmento desde el descriptor al desplazamiento para formar una dirección lineal.

Si el espacio de dirección lineal está paginado, un segundo nivel de traducción se utiliza para traducir la dirección lineal en una dirección física.

Selector de segmento

No apunta directamente al segmento, sino al descriptor de segmento que define el mismo.

- **INDEX:** selecciona uno de los 8192 descriptors en GDT o LDT. El procesador se multiplica el valor del índice en 8 (el número de bytes en un descriptor de segmento) y agrega el resultado a la base dirección del GDT o LDT
- **TI:** especifica si debe consultar la tabla GDT o LDT.
- **Requested Privilege Level (RPL):** especifica el nivel de privilegio del selector. El nivel de privilegio puede variar de 0 a 3, siendo 0 el nivel más privilegiado.

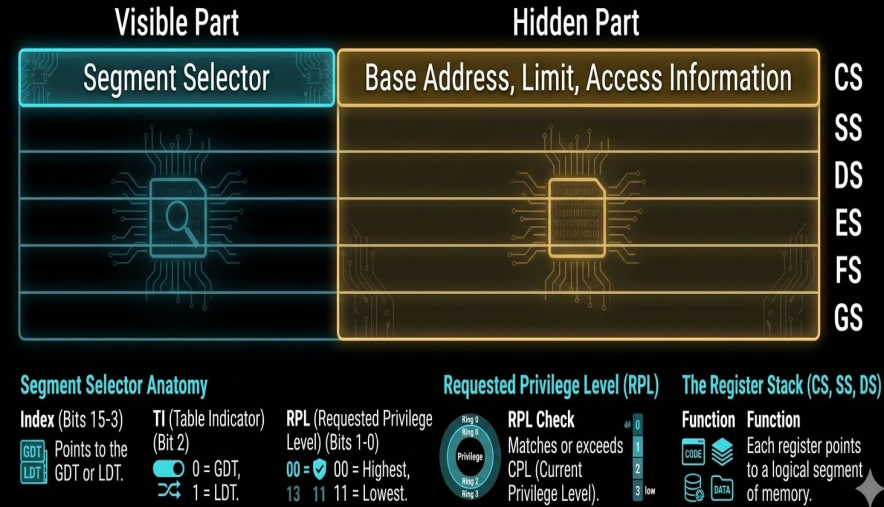


Registro de segmento

La información almacenada en caché en el registro de segmento (visible y oculto) permite al procesador traducir direcciones sin utilizar ciclos adicionales para leer la dirección base y el límite del descriptor de segmento.

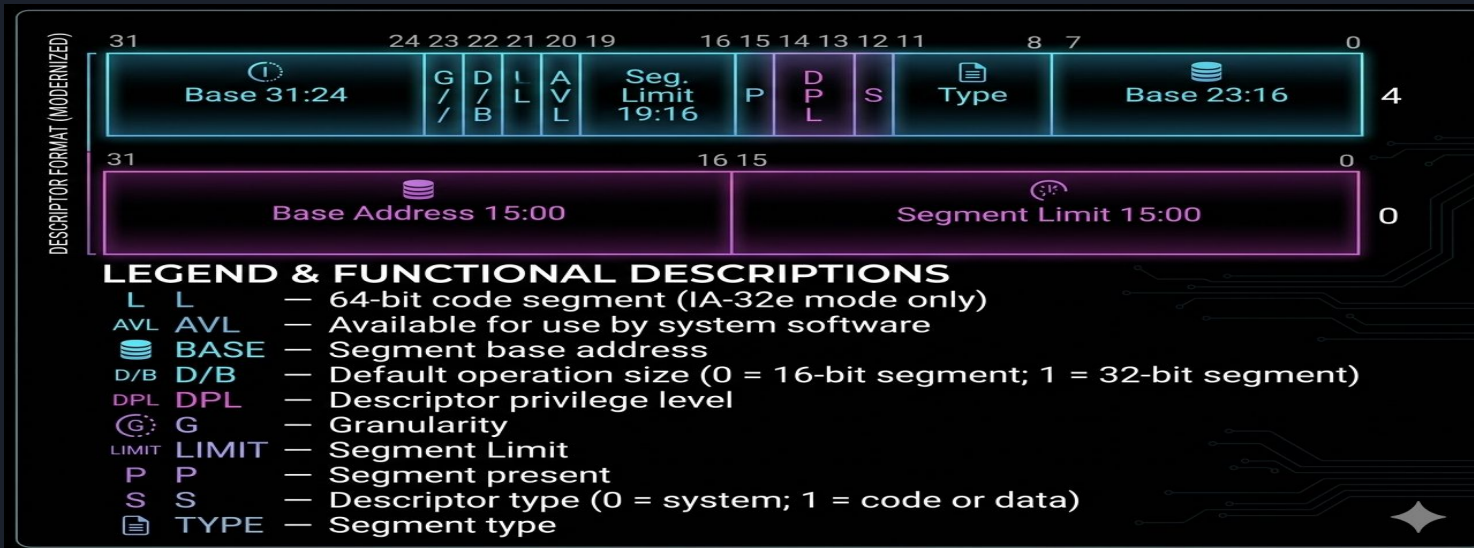
El procesador proporciona registros para almacenar hasta 6 selectores de segmento.

Cada uno de estos registros de segmento admite un tipo específico de referencia de memoria (código, pila o datos). CS code-segment, SS stack-segment, DS data-segment



Descriptor de segmento

Un descriptor de segmento es una estructura de datos en un GDT o LDT que proporciona al procesador el tamaño y la ubicación de un segmento, así como control de acceso e información de estado. Los descriptors de segmento generalmente son creados por compiladores, enlazadores, cargadores, o el sistema operativo.





Descriptor de segmento

- Base address fields, define la ubicación del byte 0 del segmento dentro del espacio de direcciones lineal de 4 GB.
- P (segment-present) flag, indica si el segmento está presente en memoria o no.
- DPL (descriptor privilege level), indica los privilegios del segmento.
- S (descriptor type) flag, especifica si el descriptor de segmento es para un segmento del sistema o para código o datos.

Code and Data Segment Types

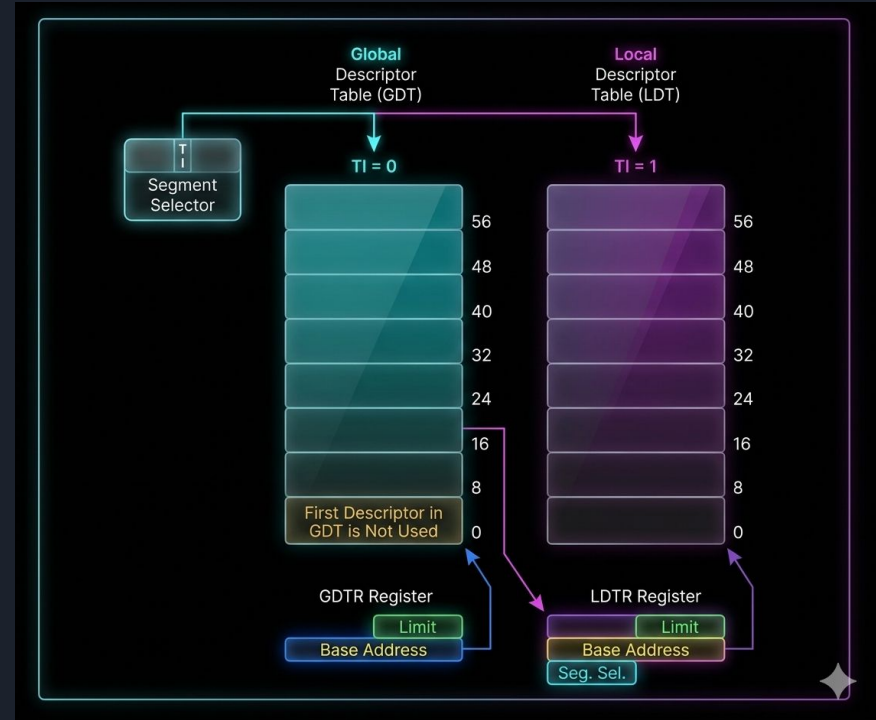
Decimal	Type Field				Descriptor Type	Description
	11	10 / E	9 / W	8 / A		
0	0	0	0	0	Data	Read-Only
1	0	0	0	1	Data	Read-Only, accessed
2	0	0	1	0	Data	Read/Write
3	0	0	1	1	Data	Read/Write, accessed
4	0	1	0	0	Data	Read/Only, expand-down
5	0	1	0	1	Data	Read-Only, expand-down, accessed
6	0	1	1	0	Data	Read/Write, expand-down
7	0	1	1	1	Data	Read/Write, expand-down, accessed
		C	R	A		
8	1	0	0	0	Code	Execute-Only
9	1	0	0	1	Code	Execute-Only, accessed
10	1	0	1	0	Code	Execute/Read
11	1	0	1	1	Code	Execute/Read, accessed
12	1	1	0	0	Code	Execute-Only, conforming
13	1	1	0	1	Code	Execute-Only, conforming, accessed
14	1	1	1	0	Code	Execute/Read, conforming
15	1	1	1	1	Code	Execute/Read, conforming, accessed

Tabla de descriptores de segmento

Es un array descriptores de segmento que puede contener hasta (8192)

2^{13} descriptores de 8 bytes. Existen dos tipos:

- GDT (tabla de descriptores globales).
- LDT (tabla de descriptores locales).





GDT y LDT

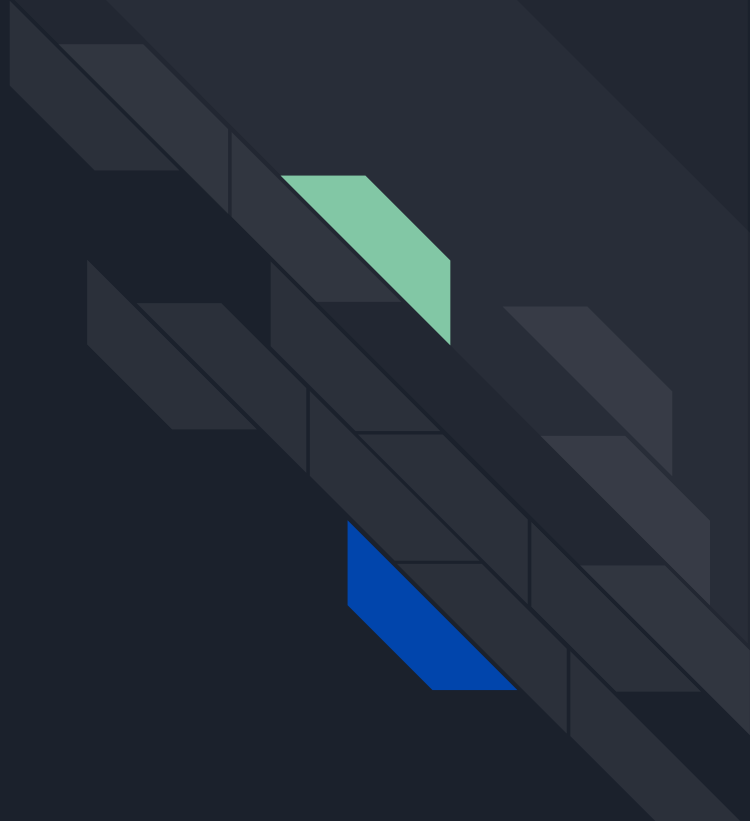
Cada sistema debe tener un GDT definida, que puede usarse para todos los programas y tareas en el sistema. Opcionalmente se pueden definir uno o más LDT.

La GDT es una estructura de datos en un espacio de direcciones lineal. La dirección lineal base y el límite de GDT debe cargarse en el registro GDTR.

La GDT debe contener un descriptor de segmento para el LDT segmento. Si el sistema admite múltiples LDT, cada uno debe tener un selector de segmento y un descriptor de segmento separados en la GDT.

La dirección lineal base, el límite y los derechos de acceso de LDT se almacenan en el registro LDTR.

Gracias!

An abstract graphic on the right side of the slide, consisting of several dark gray rectangular blocks arranged in a staggered, 3D-like pattern. Two blocks are highlighted: a light green one in the upper middle and a blue one in the lower right.